

Complementi di Programmazione

Programmazione Funzionale

CdL Informatica - Università degli studi di Modena e Reggio Emilia
AA 2024/2025

Angelo Ferrando

Paradigmi di programmazione

Avete già visto diversi paradigmi

- Imperativo
- Ad oggetti

Paradigmi di programmazione

Avete già visto diversi paradigmi

- Imperativo
 - Il programma è una serie di istruzioni che vengono eseguite una dopo l'altra
 - Il programmatore scrive le funzioni per risolvere il problema
- Ad oggetti
 - Il programma è organizzato in oggetti che esibiscono metodi
 - Incapsulano degli attributi che non vengono esibiti

Paradigmi di programmazione

Avete già visto diversi paradigmi

- Imperativo
 - Es. C
- Ad oggetti
 - Es. Java

Paradigmi di programmazione

Vi sono altri paradigmi (per esempio):

- Funzionale
- Logico

Paradigmi di programmazione

Vi sono altri paradigmi (per esempio):

- Funzionale
 - Programmazione in stile matematico
 - si dichiara che funzione utilizzare per ottenere un risultato
- Logico
 - Si esprime il problema sotto forma di vincoli
 - Si deve trovare un soluzione che soddisfi i vincoli

Paradigmi di programmazione

Vi sono altri paradigmi (per esempio):

- Funzionale
 - Es. LISP
- Logico
 - Es. Prolog

Paradigma Funzionale

E' un paradigma in stile dichiarativo. Ovvero si dichiara che valore deve assumere un dato nome.

Tipicamente ad un nome viene assegnato il risultato di una funzione.

Si dice che utilizza “espressioni” come in matematica.

Le espressioni vengono valutate per portare al risultato.

Focus su cosa risolvere.

Paradigma Funzionale

- Le Funzioni utilizzate in questo paradigma vengono definite Pure
 - Stesso input implica stesso risultato
 - Non modificano gli argomenti in input
- Non si utilizzano i cicli
 - Si utilizza la ricorsione
- Sono linguaggi di ordine superiore
 - le funzioni possono essere passate come parametri o ritornare come valore
- I nomi (che in altri linguaggi chiamiamo variabili) sono immutabili (per questo non si chiamano variabili)

Paradigma Funzionale in Python

In Python abbiamo tutti gli ingredienti per poter programmare in stile funzionale.

- Le Funzioni utilizzate in questo paradigma vengono definite Pure
 - POSSIBILE
- Non si utilizzano i cicli
 - POSSIBILE
- Sono linguaggi di ordine superiore
 - LE FUNZIONI SONO OGGETTI
- I nomi (che in altri linguaggi chiamiamo variabili) sono immutabili (per questo non si chiamano variabili)
 - POSSIBILE

Paradigma Funzionale in Python

In Python abbiamo tutti gli ingredienti per poter programmare in stile funzionale.

- Le Funzioni utilizzate in questo paradigma vengono definite Pure
 - POSSIBILE

Basta non modificare i parametri in ingresso e al limite copiarli e lavorare sulle copie

Paradigma Funzionale in Python

In Python abbiamo tutti gli ingredienti per poter programmare in stile funzionale.

- Non si utilizzano i cicli
 - POSSIBILE

Se necessario utilizzare la ricorsione

Paradigma Funzionale in Python

In Python abbiamo tutti gli ingredienti per poter programmare in stile funzionale.

- Sono linguaggi di ordine superiore
 - LE FUNZIONI SONO OGGETTI

In Python le funzioni sono oggetti come gli altri, pertanto possono essere passate come argomenti o ritornate da altre funzioni.

Paradigma Funzionale in Python

In Python abbiamo tutti gli ingredienti per poter programmare in stile funzionale.

- I nomi (che in altri linguaggi chiamiamo variabili) sono immutabili (per questo non si chiamano variabili)

Il programmatore può consapevolmente non modificare una variabile (Python).

Paradigma Funzionale in Python

Esempio classico: Fattoriale

```
def fattoriale(x):  
    return 1 if x<=1 else return x*fattoriale(x-1)
```

```
fact10 = fattoriale(10)
```

... da qui in poi non devo cambiare fact10 (immutabile)

Paradigma Funzionale in Python

Esempio classico: Fattoriale

```
def fattoriale(x):
```

```
    return 1 if x<=1 else return x*fattoriale(x-1)
```

Versione ricorsiva

Non si usano loop

```
fact10 = fattoriale(10)
```

... da qui in poi non devo cambiare fact10 (immutabile)

Paradigma Funzionale in Python

Esempio classico: Fattoriale

```
def fattoriale(x):  
    return 1 if x<=1 else return x*fattoriale(x-1)
```

Il risultato della funzione è sempre quello (dato lo stesso input)

```
fact10 = fattoriale(10)
```

... da qui in poi non devo cambiare fact10 (immutabile)

Paradigma Funzionale in Python

Esempio classico: Fattoriale

```
def fattoriale(x):
```

```
    return 1 if x<=1 else return x*fattoriale(x-1)
```

Non modifico gli argomenti (non
modifico proprio nulla)

```
fact10 = fattoriale(10)
```

... da qui in poi non devo cambiare fact10 (immutabile)

Paradigma Funzionale in Python

La ricorsione può saturare lo stack. Per questo Python ha un limite di chiamate ricorsive permesse.

Lo si può modificare con

```
import sys
```

```
sys.setrecursionlimit(<massimo_chiamate>)
```

Vantaggi della programmazione funzionale

- Maggiore facilità di debug.
 - Il risultato di una funzione è dato solamente dagli argomenti
- Funzioni tendenzialmente più semplici
 - assenza di cicli da seguire
 - assenza di flussi complessi
- Struttura più interpretabile da un compilatore
 - maggiori ottimizzazioni
 - possibili parallelizzazioni automatiche
 - lazy evaluation -> esecuzione solo quando si ha effettivamente bisogno del risultato

Svantaggi della programmazione funzionale

- Cambio di paradigma implica cambio di abitudini
 - mentalità diversa
 - struttura del codice diversa
 - Dopo un po' di allenamento comunque diventa automatico
- Poco utilizzata
 - l'imperativo è più semplice

Strumenti per la programmazione funzionale in Python

Python mette a disposizione diversi strumenti (e sintassi) utili al fine della programmazione funzionale

Alcuni di essi li abbiamo già visti senza sapere che fossero funzionali

Strumenti per la programmazione funzionale in Python

List Comprehension

Costrutto sintattico per la creazione di liste (o dizionari) senza un ciclo for classico (ma innestato dentro la lista)

[expression **for** value **in** iterable **if** condition]

Sintassi con stesse parole di un ciclo for ma la semantica è differente

Il risultato è una lista risultante da tutti i risultati di expression calcolati con ogni value nell'iterabile (se la condizione è vera)

Strumenti per la programmazione funzionale in Python

List Comprehension

```
[k*k for k in range(1, n+1)]
```

```
[k for k in range(1,n+1) if n % k == 0]
```


Strumenti per la programmazione funzionale in Python

List Comprehension

Set

```
{ k*k for k in range(1, n+1) }
```

Dictionary

```
{ k : k*k for k in range(1, n+1) }
```

Strumenti per la programmazione funzionale in Python

List Comprehension

Vi possono essere anche più for

```
[x for iter2 in iter1 for x in iter2]
```

Notare che i nomi vengono letti da sinistra verso destra

`[x for x in iter2 for iter2 in iter1]` -> Errore perchè iter2 non si sa cosa sia quando viene visto

Strumenti per la programmazione funzionale in Python

Lambda

Fondamento della programmazione funzionale.

Ogni funzione ha argomenti e risultato. Funzioni Pure

Potenzialmente un intero programma può essere scritto con sole funzioni lambda
(Turing Completo)

Strumenti per la programmazione funzionale in Python

Lambda

lambda argomenti: espressione

Questa funzione può avere un numero qualsiasi di argomenti ma solo un'espressione, che viene valutata e restituita

Strumenti per la programmazione funzionale in Python

Lambda

Esempio filter() -> filtra in base al risultato di una funzione (True tiene, False scarta)

```
filter( lambda x: (x % 2 != 0), iter1)
```

Esempio map() -> mappa il risultato di una funzione

```
map( lambda x: x*2, iter1)
```

Strumenti per la programmazione funzionale in Python

Map/Reduce

Paradigma distribuito (esiste anche un framework di Google con lo stesso nome) per la computazione dei dati.

Diviso in due fasi:

- Mappatura dei dati (filtri ed elaborazioni sui singoli elementi)
- Riduzione (aggregazione dei risultati della prima fase)

Strumenti per la programmazione funzionale in Python

Map/Reduce

Diviso in due fasi:

- Mappatura dei dati (filtri ed elaborazioni sui singoli elementi)
- Riduzione (aggregazione dei risultati della prima fase)

Essendo distribuito non si può assumere la sequenza dei dati

La riduzione può avvenire in più step.

Strumenti per la programmazione funzionale in Python

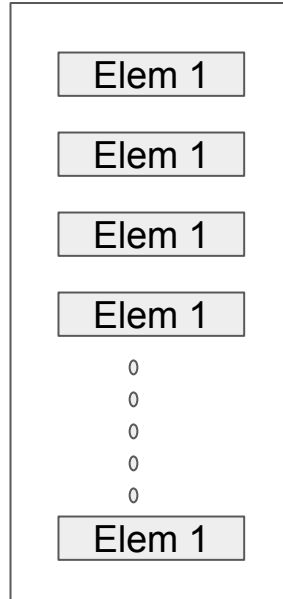
Map/Reduce

Utile in database distribuiti in cui i dati sono in nodi diversi e la computazione non può avvenire in un unico nodo.

Strumenti per la programmazione funzionale in Python

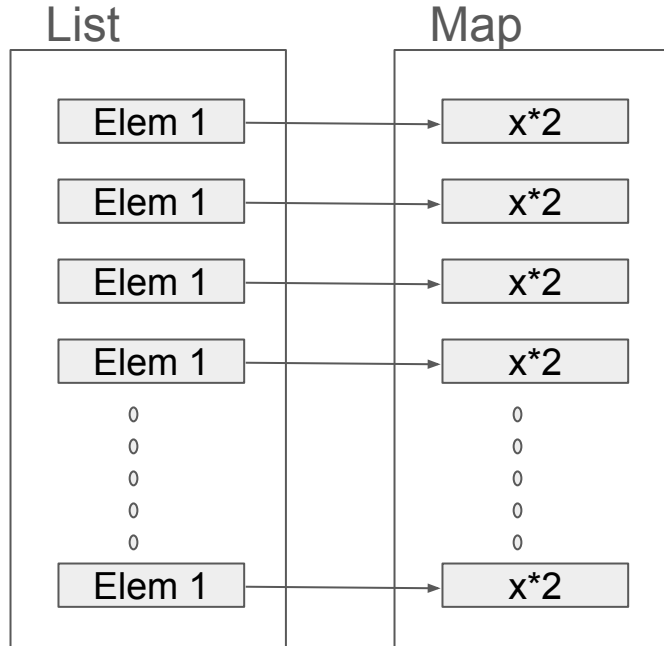
Map/Reduce

List



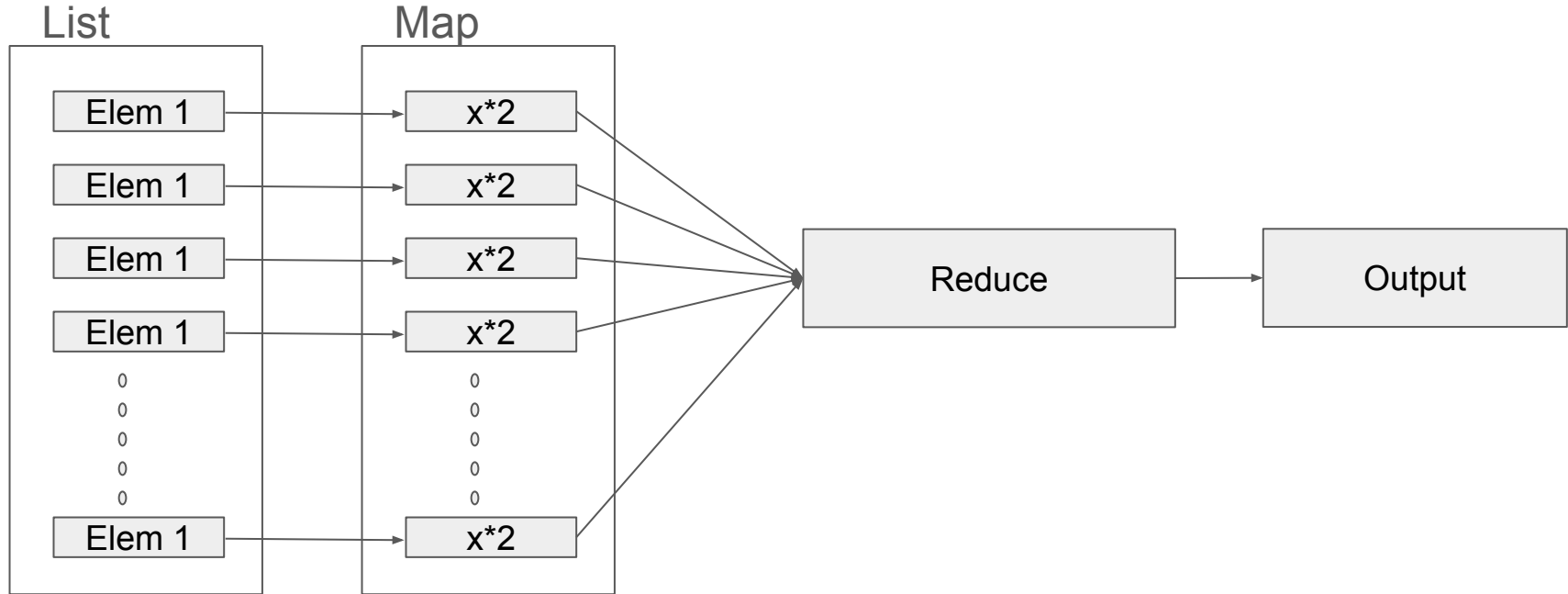
Strumenti per la programmazione funzionale in Python

Map/Reduce



Strumenti per la programmazione funzionale in Python

Map/Reduce



Strumenti per la programmazione funzionale in Python

Map/Reduce in Python

In python esiste la funzione **reduce()** che è parte della libreria **functools**.

Questa funzione esegue in locale e non in un ambiente distribuito ma è comunque utile.

Applica ripetutamente una funzione di due argomenti sugli elementi di una sequenza in modo da ridurre la sequenza ad un unico valore.

Strumenti per la programmazione funzionale in Python

Map/Reduce in Python

Applica ripetutamente una funzione di due argomenti sugli elementi di una sequenza in modo da ridurre la sequenza ad un unico valore.

Più nel dettaglio:

ad ogni iterazione il primo argomento è il risultato delle passate precedenti e il secondo è un elemento (non ancora processato) della lista.

La prima passata non ha il risultato delle passate precedenti quindi si usa il primo elemento della lista

Strumenti per la programmazione funzionale in Python

Map/Reduce in Python

La prima passata non ha il risultato delle passate precedenti quindi si usa il primo elemento della lista

Oppure

Si può specificare che valore utilizzare come primo risultato (fake)

Strumenti per la programmazione funzionale in Python

Map/Reduce in Python

Es. somma

```
reduce(lambda x,y: x+y, [1,2,3,4])
```

Es. creazione di un set da una lista

```
reduce(lambda x,y: x.union(set([y])), set())
```

Strumenti per la programmazione funzionale in Python

Map/Reduce in Python

Es. somma

```
reduce(lambda x,y: x+y, [1,2,3,4])
```

Es. creazione di un set da una lista

```
reduce(lambda x,y: x.union(set([y])), [1,2,3,4], set())
```


Strumenti per la programmazione funzionale in Python

Map/Reduce in Python

Questo paradigma è efficace sia quando la computazione può essere distribuita, sia quando può essere trattata come flusso.

Questo approccio riduce le risorse necessarie alla computazione

Strumenti per la programmazione funzionale in Python

Eval

Eval è una funzione che valuta un'espressione **passata sotto forma di stringa**

In pratica la stringa viene eseguita dall'interprete come se fosse codice del programma.

```
eval("print('ciao')")
```

Strumenti per la programmazione funzionale in Python

Eval

Eval è una funzione che valuta un'espressione **passata sotto forma di stringa**

Si possono passare anche variabili dal programma “principale”, sia come globali sia come locali al codice della stringa

```
eval("print('ciao')", globals, locals)
```

Strumenti per la programmazione funzionale in Python

Eval

a = 10

b = 20

```
eval("print(f'c: {c} d: {d}'))", {'c': a}, {'d': b})
```

Strumenti per la programmazione funzionale in Python

Eval

Il valore di ritorno di eval è il risultato dell'espressione valutata

`eval("1+1") -> 2`

Strumenti per la programmazione funzionale in Python

Eval

Eval può valutare qualsiasi espressioni con tutti i problemi relativi alla sicurezza.

E' buona norma non far valutare stringhe generiche oppure settare accuratamente globals e locals in modo da non dare accesso a funzioni critiche

Strumenti per la programmazione funzionale in Python

Exec

`exec()` è molto simile ad `eval` ma più completo

- Può eseguire anche istruzioni (non solo espressioni)
- Può eseguire codice compilato

Strumenti per la programmazione funzionale in Python

Exec

`exec()` è molto simile ad `eval` ma più completo

- Può eseguire anche istruzioni (non solo espressioni)

Es.

```
exec("import subprocess")
```


Strumenti per la programmazione funzionale in Python

Exec

`exec()` è molto simile ad `eval` ma più completo

- Può eseguire codice compilato

Anche in python vi è un formato intermedio (bytecode). Ci si riferisce a questo con compilato

Strumenti per la programmazione funzionale in Python

Exec

`exec()` è molto simile ad `eval` ma più completo

- Può eseguire codice compilato

Anche in python vi è un formato intermedio (bytecode). Ci si riferisce a questo con compilato

`exec(compiled)`

Strumenti per la programmazione funzionale in Python

Compile

`compile()` compila una stringa (o un file) in bytecode

`compile(source, file, mode)`

Strumenti per la programmazione funzionale in Python

Compile

`compile()` compila una stringa (o un file) in bytecode

`compile(source, file, mode)`

stringa da compilare

`compile("print()", "<string>", mode)`

Strumenti per la programmazione funzionale in Python

Compile

`compile()` compila una stringa (o un file) in bytecode

`compile(source, file, mode)`

file da compilare

`compile("", "source.py", mode)`

Strumenti per la programmazione funzionale in Python

Compile

`compile()` compila una stringa (o un file) in bytecode

`compile(source, file, mode)`

Modalità:

‘exec’ se contiene delle istruzioni

‘eval’ se contiene una singola espressione

‘single’ se contiene una singola istruzione interattiva

Strumenti per la programmazione funzionale in Python

Functools

Modulo utile per la programmazione funzionale in Python

Fornisce funzioni di ordine superiore per eseguire operazioni comuni nella programmazione funzionale

Fornisce anche due classi per rappresentare funzioni: **partial** e **partialmethod**

Strumenti per la programmazione funzionale in Python

Functools

Partial rappresenta una funzione in cui alcuni argomenti sono definiti prima di chiamarla

```
def somma(x, y):  
    return x+y
```

```
sommapartial = partial(somma, y=1)
```


Strumenti per la programmazione funzionale in Python

Functools

```
def somma(x, y):  
    return x+y
```

```
sommapartial = partial(somma, y=1)
```

Quando si chiama **sommapartial** basterà passare solo un argomento. La y è stata definita prima.

Strumenti per la programmazione funzionale in Python

Functools

`partialmethod()` è simile a `partial()` ma lavora sui metodi di una classe.

E' possibile definire un metodo parziale che richiama un altro metodo ma con dei parametri preimpostati

Strumenti per la programmazione funzionale in Python

Functools

```
class A:
```

```
    def print_str(self, s):
```

```
        print(s)
```

```
    print_ciao = partialmethod(print_str, s='ciao')
```

Strumenti per la programmazione funzionale in Python

Functools

cmp_to_key trasforma una funzione di comparazione in una funzione chiave

Es.

`sorted()` ordinava i valori di una lista

`sorted([3,4,6,2,8,1])`

Vi era la possibilità di specificare una funzione chiave

`sorted([(1,2), (2,1)], key=lambda x:x[1])`

Strumenti per la programmazione funzionale in Python

Functools

cmp_to_key trasforma una funzione di comparazione in una funzione chiave

Es.

Vi era la possibilità di specificare una funzione chiave

```
sorted([(1,2), (2,1)], key=lambda x:x[1])
```

In questo caso ad ogni elemento è applicata la funzione key. E poi gli elementi vengono ordinati usando questo risultato

Strumenti per la programmazione funzionale in Python

Functools

cmp_to_key trasforma una funzione di comparazione in una funzione chiave

Es.

Vi era la possibilità di specificare una funzione chiave

```
sorted([(1,2), (2,1)], key=lambda x:x[1])
```

Notare che non avviene nella funzione lambda il confronto tra due elementi.
(necessario all'ordinamento)

Strumenti per la programmazione funzionale in Python

Functools

cmp_to_key trasforma una funzione di comparazione in una funzione chiave

Es.

supponiamo di avere oggetti più complessi da ordinare

class Data:

```
def __init__(self, anno, mese, giorno):
```

```
    self.anno = anno
```

```
    ...
```

Strumenti per la programmazione funzionale in Python

Functools

cmp_to_key trasforma una funzione di comparazione in una funzione chiave

class Data:

```
def __init__(self, anno, mese, giorno):
```

```
    self.anno = anno
```

```
    ...
```

Non è banale restituire una chiave che sorted possa utilizzare per l'ordinamento

Strumenti per la programmazione funzionale in Python

Functools

cmp_to_key trasforma una funzione di comparazione in una funzione chiave

Possiamo però scrivere una funzione di confronto tra due date

Strumenti per la programmazione funzionale in Python

Functools

```
def cmp_date(x,y):
```

```
    if x.anno < y.anno
```

```
        return -1
```

```
    elif x.anno > y.anno:
```

```
        return 1
```

```
    else: #caso stesso anno
```

```
        if x.mese < y.mese:
```

```
            retrun -1
```

```
        elif x.mese > y.mese:
```

```
            return 1 ...
```

E poi utilizzare `cmp_to_key` per avere una funzione chiave

```
cmp_to_key(cmp_date)
```

Strumenti per la programmazione funzionale in Python

Functools

total_ordering è un decoratore per classi che fornisce i metodi di confronto per quella classe.

Basandosi sul metodo `__eq__` e uno tra gli altri metodi di confronto (`__lt__()`, `__le__()`, `__gt__()`, `__ge__()`) inferisce gli altri

Strumenti per la programmazione funzionale in Python

Functools

Quindi basterà implementare due metodi per avere gli altri

@total_ordering

class A:

```
    def __eq__(self, other):
```

```
        ...
```

```
    def __lt__(self, other):
```

```
        ...
```

Strumenti per la programmazione funzionale in Python

Functools

LRU_cache è un decoratore che fa in modo di evitare la chiamata ad una funzione se è già stata invocata con gli stessi argomenti.

Memorizza il risultato e ritorna direttamente quello senza rieseguire

La funzione deve essere deterministica!

Strumenti per la programmazione funzionale in Python

Functools

LRU_cache è un decoratore che fa in modo di evitare la chiamata ad una funzione se è già stata invocata con gli stessi argomenti.

```
@lru_cache()
```

```
def somma(x,y):  
    return x+y
```

Se invoco `somma(1,2)` più volte la funzione verrà eseguita una sola volta

Strumenti per la programmazione funzionale in Python

Functools

LRU_cache è un decoratore che fa in modo di evitare la chiamata ad una funzione se è già stata invocata con gli stessi argomenti.

```
@lru_cache(maxsize=10)
```

```
def somma(x,y):
```

```
    return x+y
```

Numero massimo di
memorizzazioni

Se invoco `somma(1,2)` più volte la funzione verrà eseguita una sola volta

Strumenti per la programmazione funzionale in Python

Itertools

Itertools è un modulo che fornisce funzioni per generare sequenze di dati iterabili.

Molto utile in programmazione funzionale

Strumenti per la programmazione funzionale in Python

Itertools

iteratori infiniti

count(10) -> 10,11,12 ...

cycle([1,2,3]) -> 1, 2, 3, 1, 2, 3, ...

repeat('a') -> a,a,a,a...

Strumenti per la programmazione funzionale in Python

Itertools

iteratori che modificano la sequenza

accumulate([1,2,3,4]) -> 1 3 6 10

chain('abc', 'def') -> a b c d e f

zip_longest('ABC', 'xyz') -> Ax By Cz

starmap(pow, [(1,2), (3,4)]) -> applica pow(*x) -> 1 81

Strumenti per la programmazione funzionale in Python

Itertools

iteratori combinatori

permutation([1,2,3]) -> tutte le permutazioni possibili

combination([1,2,3]) -> tutte le combinazioni possibili